

# CI/CD

CI/CD



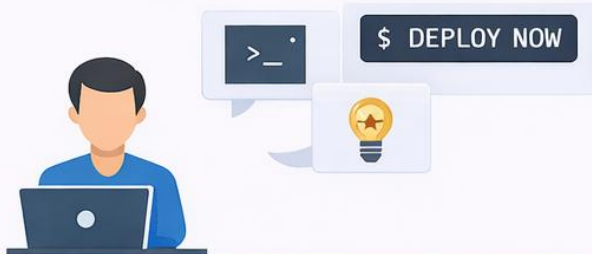
## 배포 자동화(Automation)

사람이 하던 반복적인 배포 작업을 도구가 대신, 정해진 규칙에 따라 자동으로 수행하는 것을 의미합니다.

### What is Deployment Automation?

Deployment automation means tools replace repetitive deployment tasks, performing them automatically based on predefined rules.

#### Before Automation



#### Before Automation

- ✓ Enter commands manually
- ✓ Proceed by memory
- ⚠ High chance of mistakes

#### With Automation



#### With Automation

- ✓ Triggered by preset events
- ✓ Executes predefined steps
- ✓ Runs automatically without human intervention

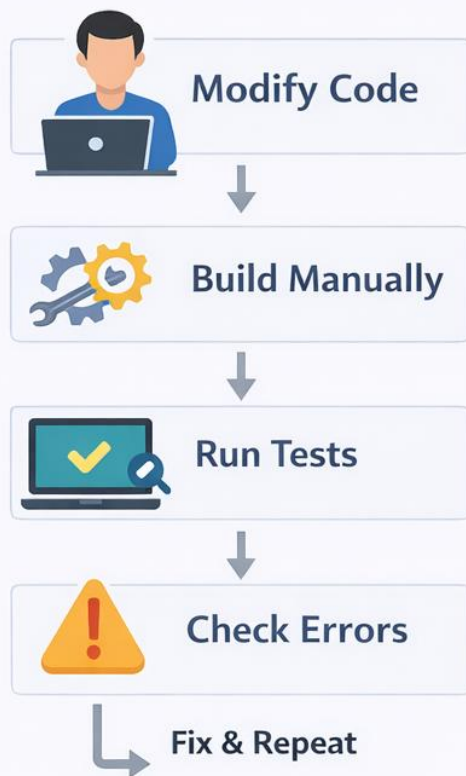
VS

This is Deployment Automation.

## 자동화와 CI/CD

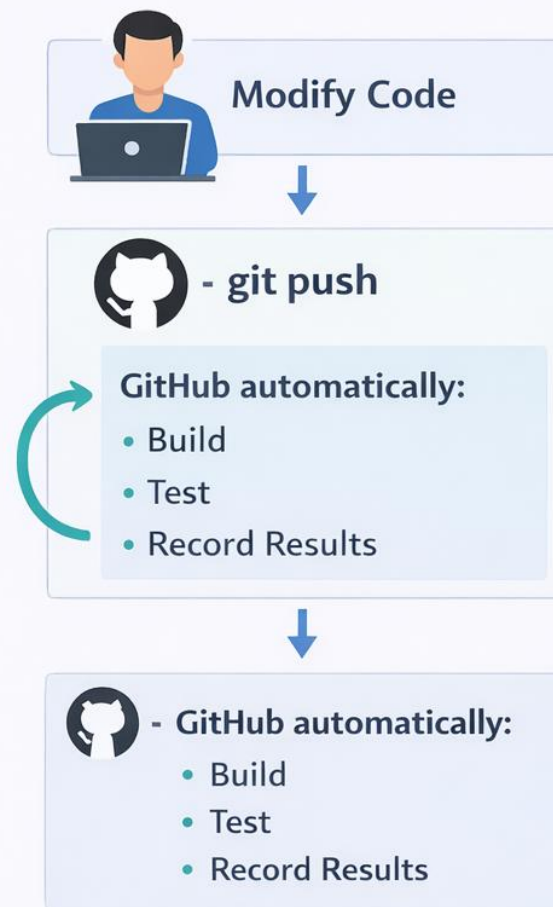
기존 방식은 수동 빌드/배포이다 보니 사람의 실수, 배포 시간 오래 걸림, 테스트 누락 가능성이 높습니다.

### Without Automation



VS

### With Automation (CI/CD)

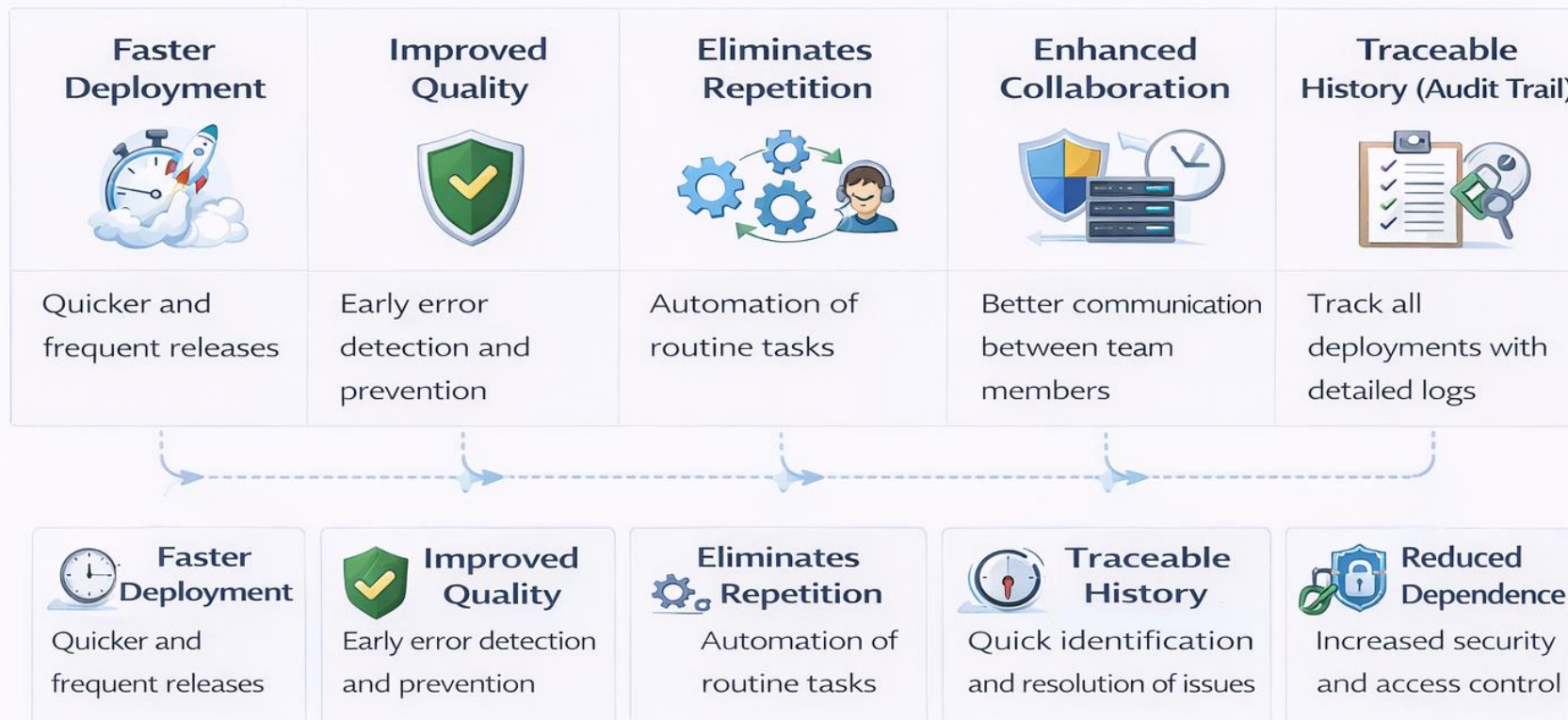


## CI/CD의 장점

코드 변경부터 테스트와 배포까지를 자동화하여 개발 속도는 높이고 오류와 배포 리스크는 줄여 주는 것입니다.

### Benefits of CI/CD

Automates code changes from testing to deployment, speeding up development while reducing errors and deployment risks.



## ≡ CI/CD 도구 예시

CI/CD 도구는 개발부터 배포까지를 자동화해주는 도구들입니다.

### CI CD Tools

#### Examples of CI/CD Tools

##### GitHub Actions



##### GitHub Actions:

CI/CD tool that automates build, test, and deploy based on GitHub repository events

##### Jenkins



Open-source CI/CD server that supports flexible pipelines through various plugins

##### GitLab CI



CI/CD tool integrated with GitLab, offering code management to deployment solutions

##### CircleCI



Cloud-based CI/CD service supporting fast builds and parallel execution



## ≡ CI/CD 배포 대상

CI/CD 도구들의 실행 결과물은 서버 · 컨테이너 · 클라우드에 자동으로 배포됩니다.

배포 대상

### Examples of CI/CD Deployment Targets

Server (Linux)



**Server (Linux):**

Deploy application directly  
onto Linux servers



Docker



**Docker**

Package applications into  
containers for consistent  
environments



Cloud (AWS, OCI, etc.)



**Cloud (AWS, OCI, etc.)**

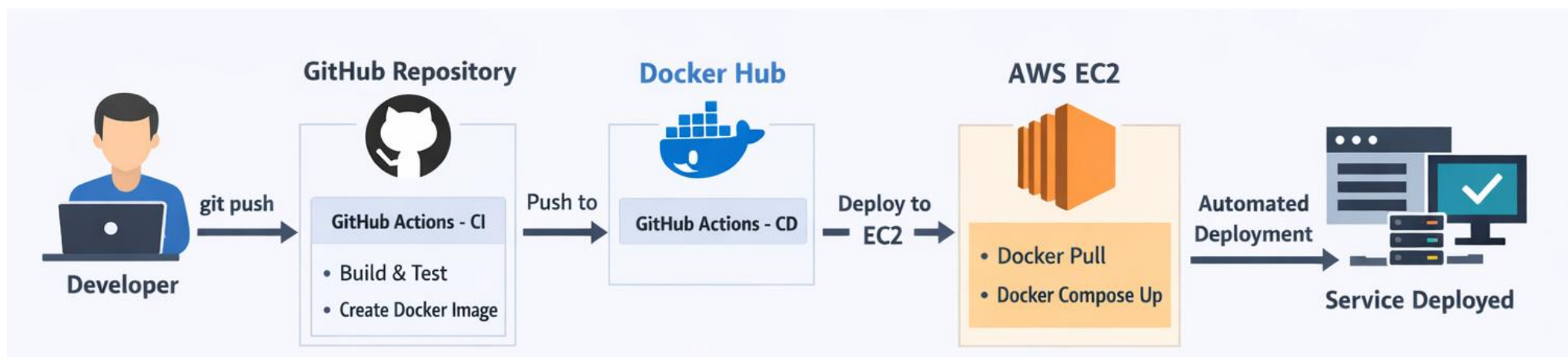
Leverage cloud infrastructure  
for scalability and reliability



## CI/CD란?

개발자가 GitHub에 코드를 푸시하면 CI/CD 파이프라인을 통해 빌드되어 AWS EC2에 자동으로 배포됩니다.

### 전체 흐름도



### CI/CD

CI/CD = 코드 변경 → 자동 빌드 → 자동 테스트 → 자동 배포

사람이 하던 반복 작업을 자동화하여 빠르고 안전하게 소프트웨어를 배포하는 방식입니다.

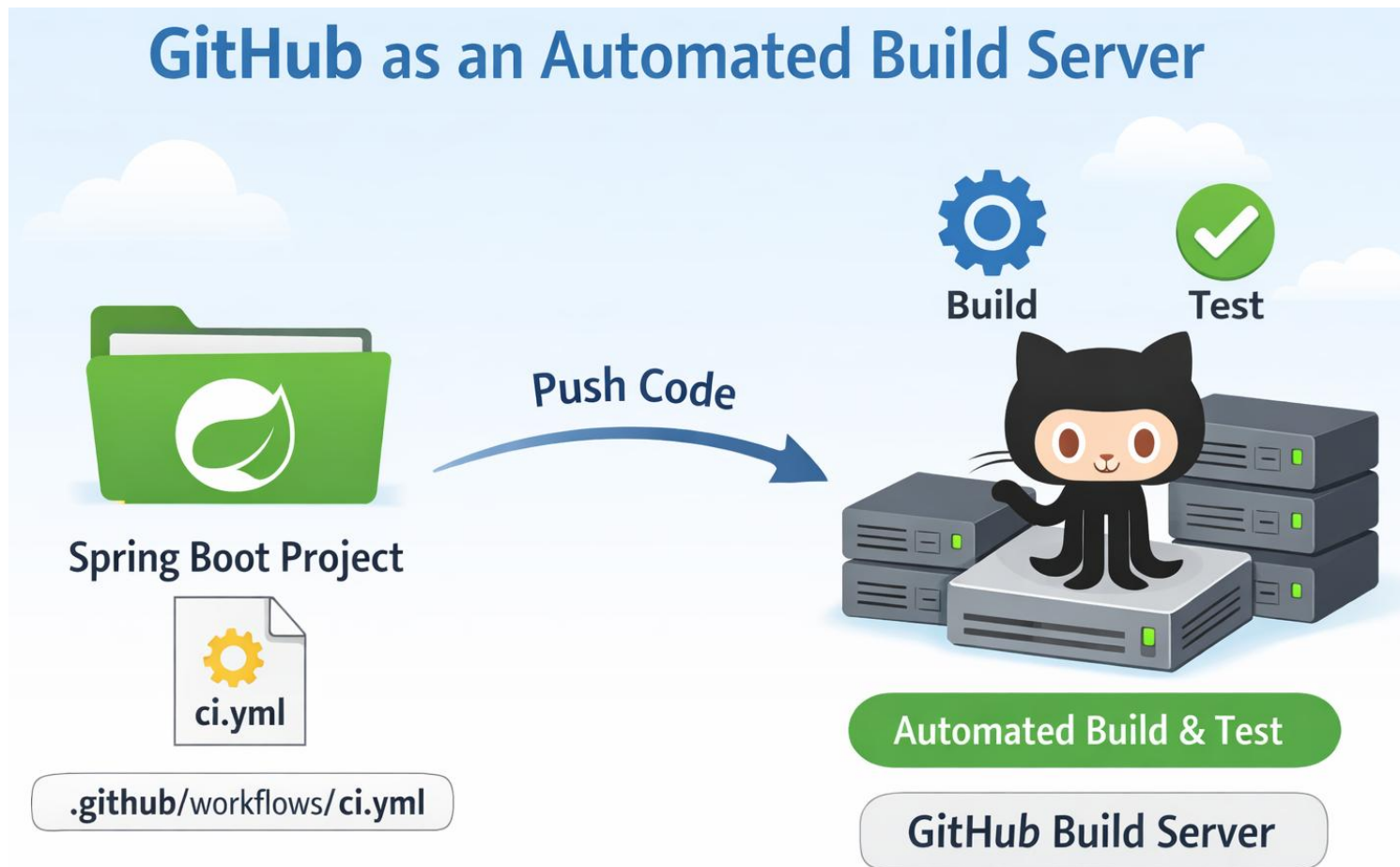
개발 속도 향상 + 오류 감소 + 안정적인 배포

# CI 구현하기

## CI(Continuous Integration)

로컬 프로젝트에 ci.yml 파일을 작성하고 Push를 하게 되면, GitHub가 자동으로 Test/Build 작업을 진행합니다.

GitHub = 자동 빌드 서버

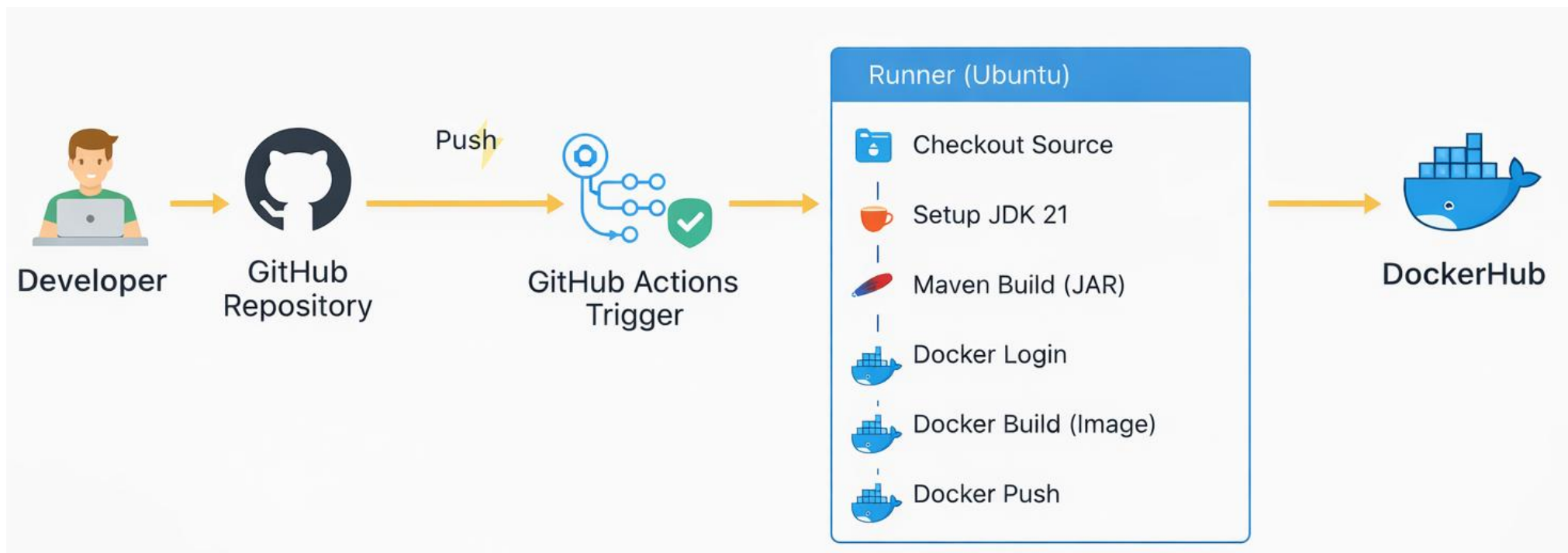




## CI(Continuous Integration)

지속적 통합은 개발자가 코드를 병합하고자 할 때, 빌드/테스트 등을 자동으로 검사해 주는 기능입니다.

### CI 전체 흐름 다이어그램



## ≡ CI(Continuous Integration)

지속적 통합은 개발자가 코드를 병합하고자 할 때, 빌드/테스트 등을 자동으로 검사해 주는 기능입니다.

CI 동작 흐름

### CI Process Overview

| Step | Step                                                                                                                                            |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    |  <b>Checkout Code</b><br>Fetch source code from GitHub        |
| 2    |  <b>Set Up JDK</b><br>Spring Boot runtime environment        |
| 3    |  <b>Build</b><br>Gradle / Maven                               |
| 4    |  <b>Test</b><br>Run unit tests (if any)                      |
| 5    |  <b>Create Docker Image</b><br>Package executable app       |
| 6    |  <b>Push to Docker Hub</b><br>Push image to Docker registry |

## ≡ CD(Continuous Delivery / Deployment)

코드의 변경이 테스트를 거쳐 자동으로 서비스에 배포되도록 하는 방식입니다.

CD 동작 흐름

### CD Process Overview

Deployment automation means tools replace repetitive deployment tasks, performing them automatically based on predefined rules.

| Step | Step                                                                                                                                                |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    |  <b>Deployment Trigger</b><br>After CI success                     |
| 2    |  <b>Connect to EC2</b><br>SSH                                      |
| 3    |  <b>Pull Image</b><br>Docker Hub                                  |
| 4    |  <b>Restart Containers</b><br>Zero-downtime or warm restart      |
| 5    |  <b>Service Reflects Changes</b><br>Users can access immediately |

## ≡ CD(Continuous Delivery / Deployment)

코드의 변경이 테스트를 거쳐 자동으로 서비스에 배포되도록 하는 방식입니다.

### 두 가지 의미

#### Continuous Delivery

- 배포 준비 상태까지 자동
- 실제 배포는 사람이 버튼 클릭

#### Continuous Deployment

- 테스트 통과 → 자동 배포

### CD의 개념

예시 :

빌드 성공

→ 테스트 통과

→ 서버에 자동 배포

## ≡ CI vs CD 비교

CI는 코드 품질을 자동으로 확인하는 단계이고, CD는 이를 실제 서비스에 배포하는 단계입니다.

### CI vs CD 비교

| 구분    | CI             | CD                   |
|-------|----------------|----------------------|
| 목적    | 코드 품질 검증/확인    | 서비스 자동 배포            |
| 실행 위치 | GitHub Actions | GitHub Actions → EC2 |
| 시점    | 코드 병합 시        | 테스트 통과 후             |
| 결과물   | Docker Image   | 실행 중인 서비스            |
| 핵심    | 빌드, 테스트        | 배포 자동화               |



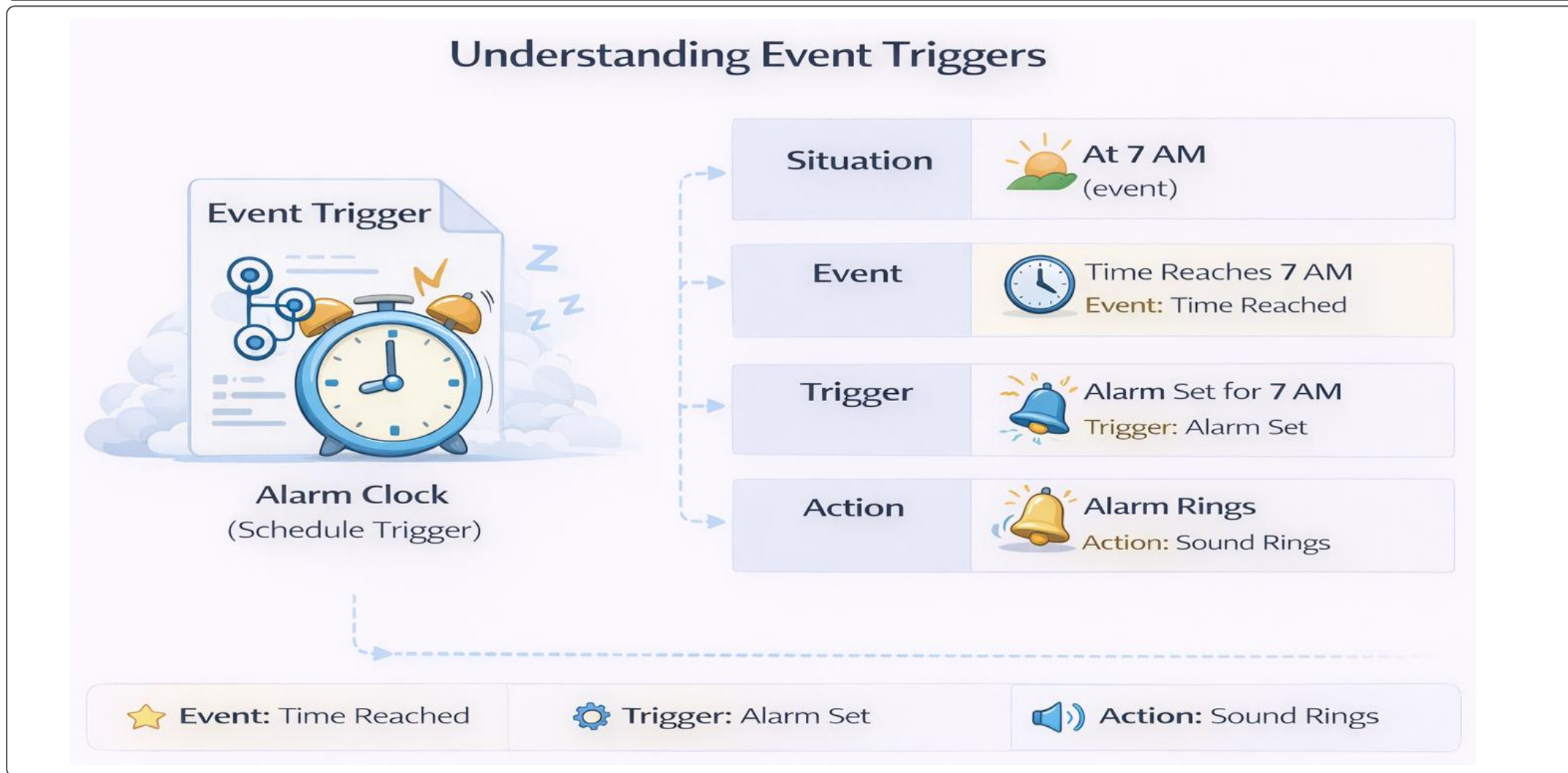
# CI/CD

GitHub Actions



## Event Trigger

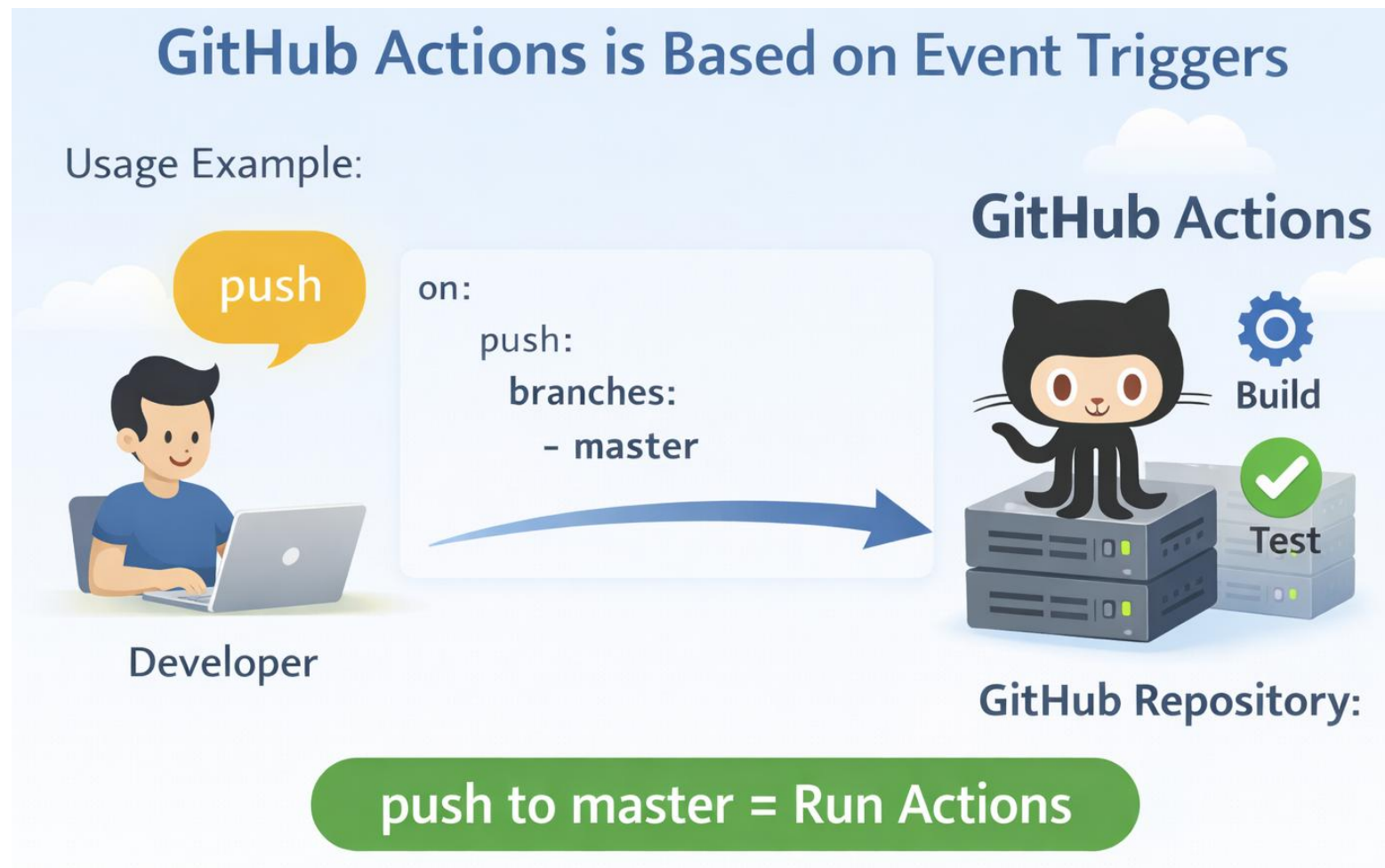
Timer는 정해진 시간이 되면 알람 설정에 의해 자동으로 소리가 울리게 됩니다.



# ≡ GitHub Actions

## ≡ GitHub Actions 개요

GitHub 저장소에서 코드 변경이 발생할 때 빌드, 테스트, 배포를 자동으로 실행해 주는 CI/CD 자동화 도구입니다.



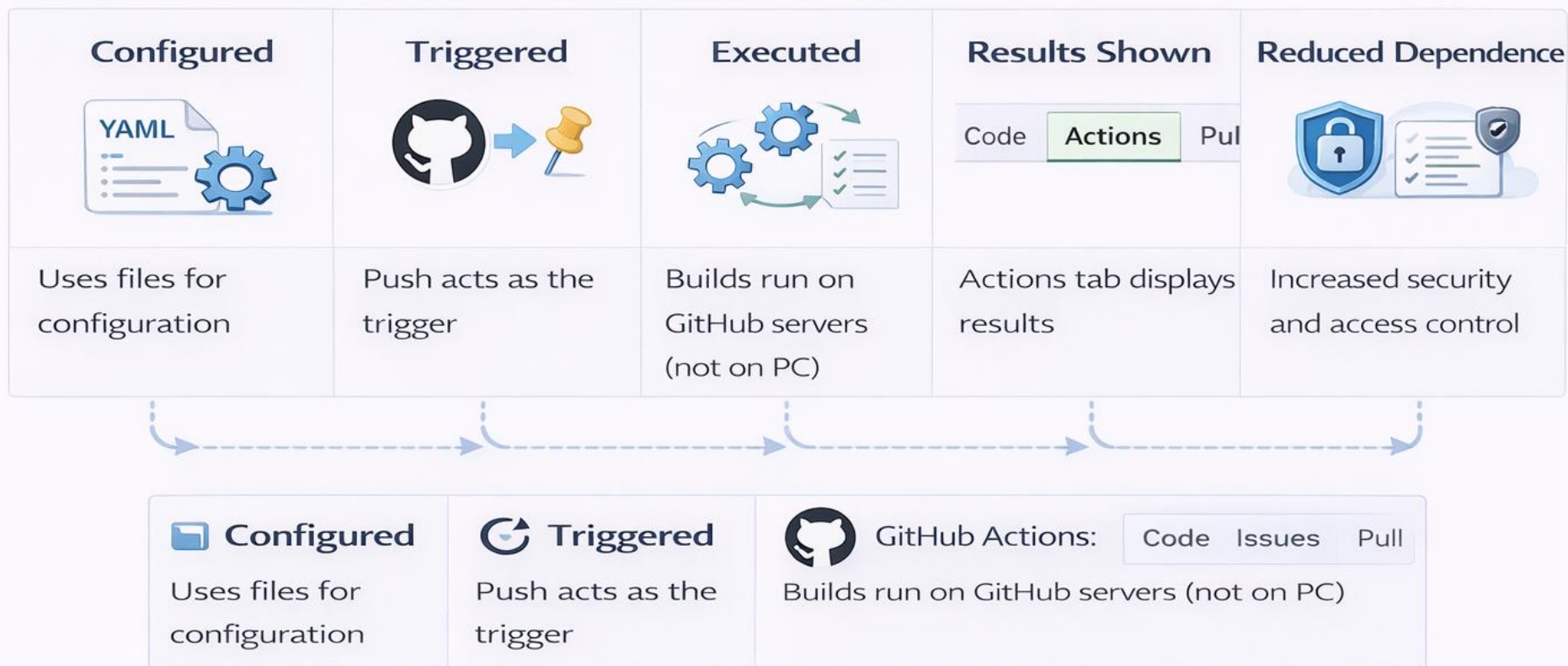
# ≡ GitHub Actions

## ≡ GitHub Actions의 핵심 개념

GitHub 저장소에서 코드 변경이 발생할 때 빌드, 테스트, 배포를 자동으로 실행해 주는 CI/CD 자동화 도구입니다.

### Core Concepts of GitHub Actions

Automates code changes from testing to deployment, speeding up development while reducing errors and deployment risks.



# ≡ GitHub Actions

## ≡ GitHub Actions Tab

GitHub Repository의 Actions Tab에서 다음과 같은 Workflow 정보를 확인할 수 있습니다.

The screenshot shows the GitHub Actions interface for the repository `seoljinuk / cicd`. The **Actions** tab is selected in the top navigation bar. The left sidebar shows the **Actions** section with a **New workflow** button and a list of workflow runs. The main area displays **All workflows** with a search bar and a table of workflow runs.

| Event                                                                           | Status | Branch | Actor             |
|---------------------------------------------------------------------------------|--------|--------|-------------------|
| ✓ add github actions ci                                                         |        | master |                   |
| Spring Boot CI #1: Commit <code>ef684a0</code> pushed by <code>seoljinuk</code> |        |        |                   |
|                                                                                 |        |        | 2 minutes ago ... |
|                                                                                 |        |        | 31s               |

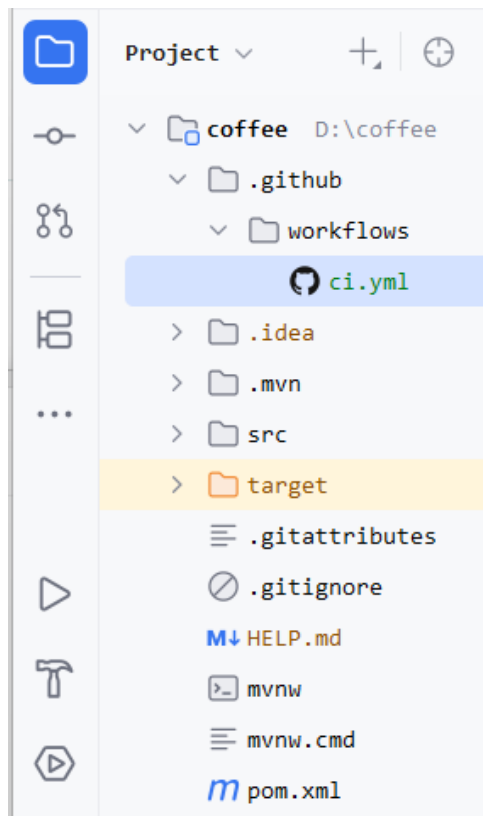
# GitHub Actions

## 설정용 파일

ci.yml 파일은 GitHub Actions에서 사용하는 CI(Continuous Integration) 설정용 파일입니다.

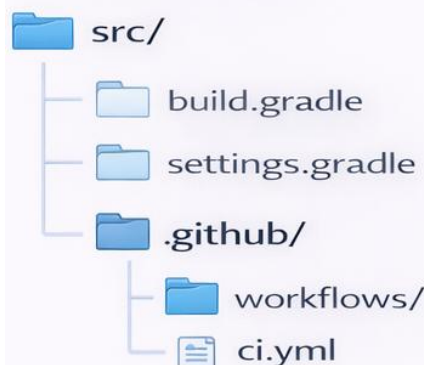
폴더 구조

이 파일은 GitHub에게 언제, 무엇을, 어떻게 자동 실행할지 알려주는 설명서 파일입니다.

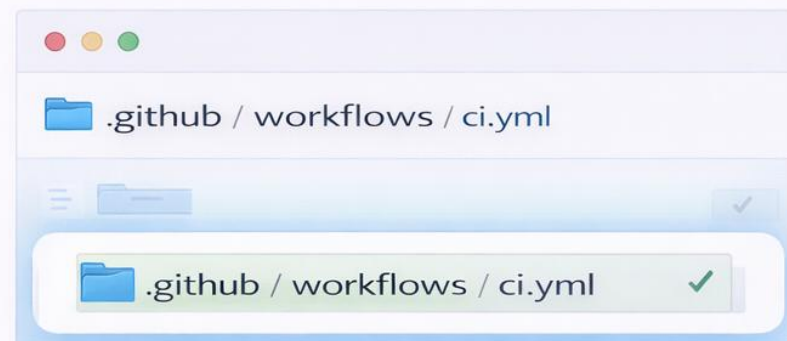


### Path of the ci.yml File

(Project Root)



.github / workflows / ci.yml

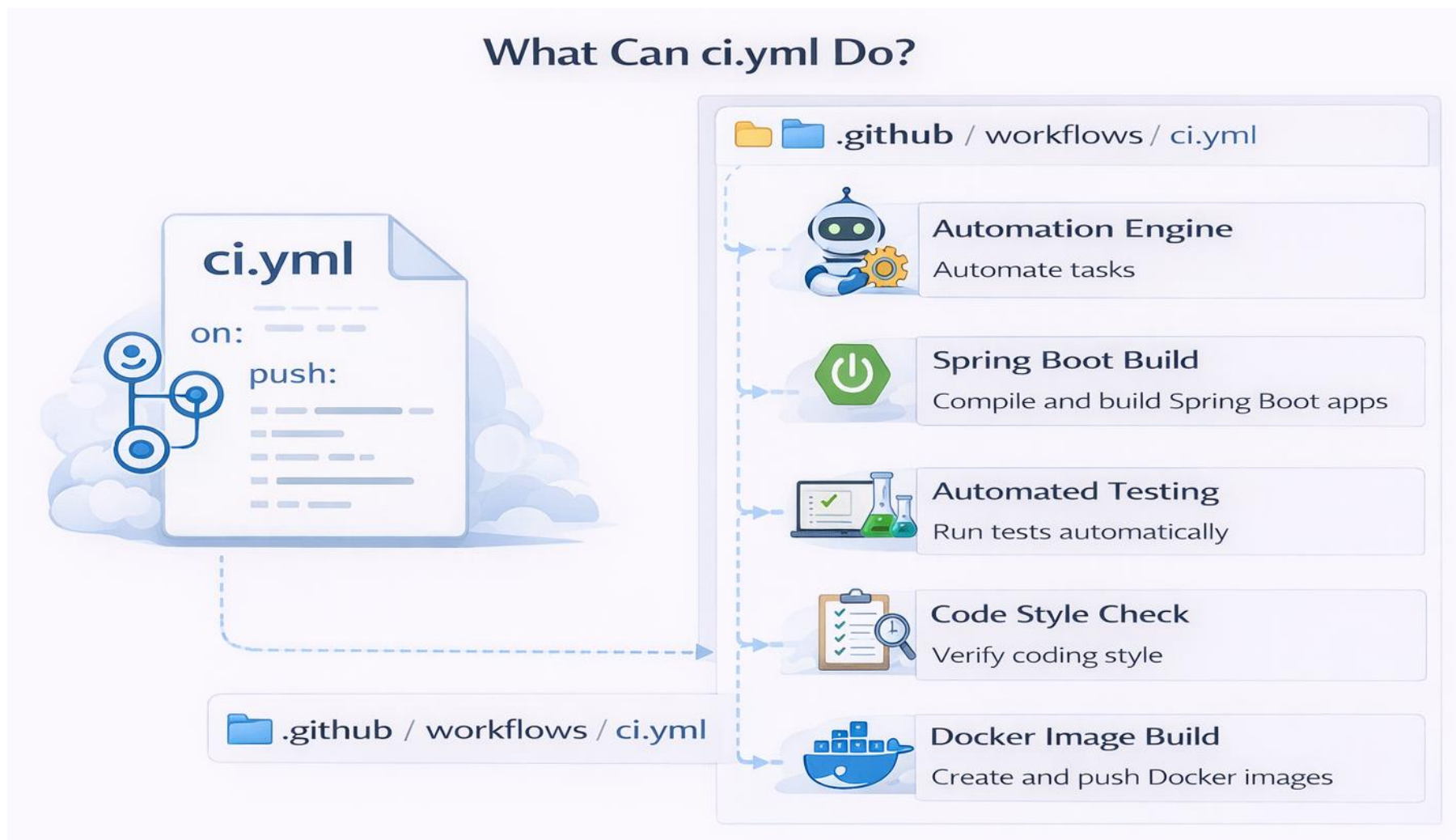




# ≡ GitHub Actions

## ≡ 설정용 파일

CI(Continuous Integration) 설정용 파일이 할 수 있는 역할은 다음과 같습니다.





## 설정용 파일

ci.yml 파일에 정의할 수 있는 내용들은 다음과 같습니다.

### 정의 내용

### Understanding the ci.yml File

The ci.yml file is a configuration file used in GitHub Actions for Continuous Integration (CI).



| When to Run                                                                                                                                                                   | Which Environment                                                                                                                                                              | What to Do                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <ul style="list-style-type: none"><li>• push event</li><li>• pull request event</li></ul> |  <ul style="list-style-type: none"><li>• Ubuntu</li><li>• Windows</li><li>• macOS</li></ul> |  <ul style="list-style-type: none"><li>• Checkout code</li><li>• Build</li><li>• Test</li></ul> |



 **ci.yml**   
Uses files for configuration

 **GitHub Actions:**   ☒ L2... 

- Ubuntu
- Windows
- macOS

- munk
- macOS

## ≡ 설정용 파일

ci.yml 파일은 GitHub Actions에서 사용하는 CI(Continuous Integration) 설정 파일입니다.

### CI 단계별 설명 표

| 구분       | Step 이름                     | 주석 내용 요약                     | 역할 / 의미                           |
|----------|-----------------------------|------------------------------|-----------------------------------|
| 워크플로우 식별 | name: Spring Boot CI        | GitHub Actions 화면에 표시되는 글자   | Actions 탭에서 이 CI를 구분하기 위한 표시용 이름  |
| 트리거 조건   | on: push → branches: master | 기본 브랜치 이름 확인                 | master 브랜치에 push 될 때 CI 자동 실행     |
| 실행 환경    | runs-on: ubuntu-latest      | Runner(가상 서버) 지정             | GitHub가 제공하는 Ubuntu 가상 머신에서 실행    |
| 소스 준비    | Checkout source             | GitHub 저장소 코드를 Runner에 clone | CI 서버에 프로젝트 소스 코드 다운로드            |
| 실행 환경 준비 | Set up JDK 21               | Java 21 Temurin 설치 및 설정      | Spring Boot 3.x에 맞는 자바 실행 환경 구성   |
| 캐시 설정    | cache: maven                | Maven 의존성 캐시 적용              | .m2 캐시로 빌드 속도 향상 및 안정성 확보         |
| 빌드 단계    | Build with Maven            | 컴파일만 하고 테스트는 실행 안 함          | target/*.jar 생성(CI 속도 우선 전략)      |
| 인증 단계    | Login to DockerHub          | DockerHub 로그인(Secrets 사용)    | CI 서버에서 이미지 push 권한 확보            |
| 이미지 생성   | Build Docker Image          | Dockerfile을 이용한 이미지 생성       | Spring Boot JAR → Docker 이미지로 패키징 |
| 이미지 배포   | Push Docker Image           | DockerHub에 이미지 업로드           | CD 서버에서 pull 가능하도록 이미지 저장소에 업로드   |

## ≡ 설정용 파일

CD(Continuous Deployment) 설정용 파일이 할 수 있는 역할은 다음과 같습니다.

### Key Roles of cd.yml File



Automate Server Connection (SSH)



Automate Docker Image Deployment



Stop and Restart Containers



Deploy Application



Check Service Status (Health Check)



Rollback or Alert on Deployment Failure



Automate Production / Staging Environment Updates

## 설정용 파일

cd.yml 파일에 정의할 수 있는 내용들은 다음과 같습니다.

### cd.yml File

Defines Continuous Deployment (CD) configuration in GitHub Actions

| When to Run                                                                                                                                                                                                                                                       | Where to Run                                                                                                                                                                                                                                                                | What to Do                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>When to Run</b> <ul style="list-style-type: none"><li>• On CI workflow success</li><li>• On push to specific branches</li><li>• Manual trigger (workflow_dispatch)</li></ul> |  <b>Where to Run</b> <ul style="list-style-type: none"><li>• ubuntu-latest (default)</li><li>• Self-hosted runner (actual deployment server)</li><li>• Specific OS environment</li></ul> |  <b>Connect to server (SSH)</b><br> <b>Pull Docker image</b><br> <b>Restart containers</b><br> <b>Deploy application</b> |
|  <b>When to Run</b> <ul style="list-style-type: none"><li>• On CI workflow success</li><li>• On push to specific branches</li></ul>                                            |  <b>ubuntu-latest</b> <ul style="list-style-type: none"><li>• Self-hosted runner (actual deployment server)</li><li>• Specific OS environment</li></ul>                                |  <b>Check service status</b>                                                                                                                                                                                                                                                                                                                                                     |

# CI/CD

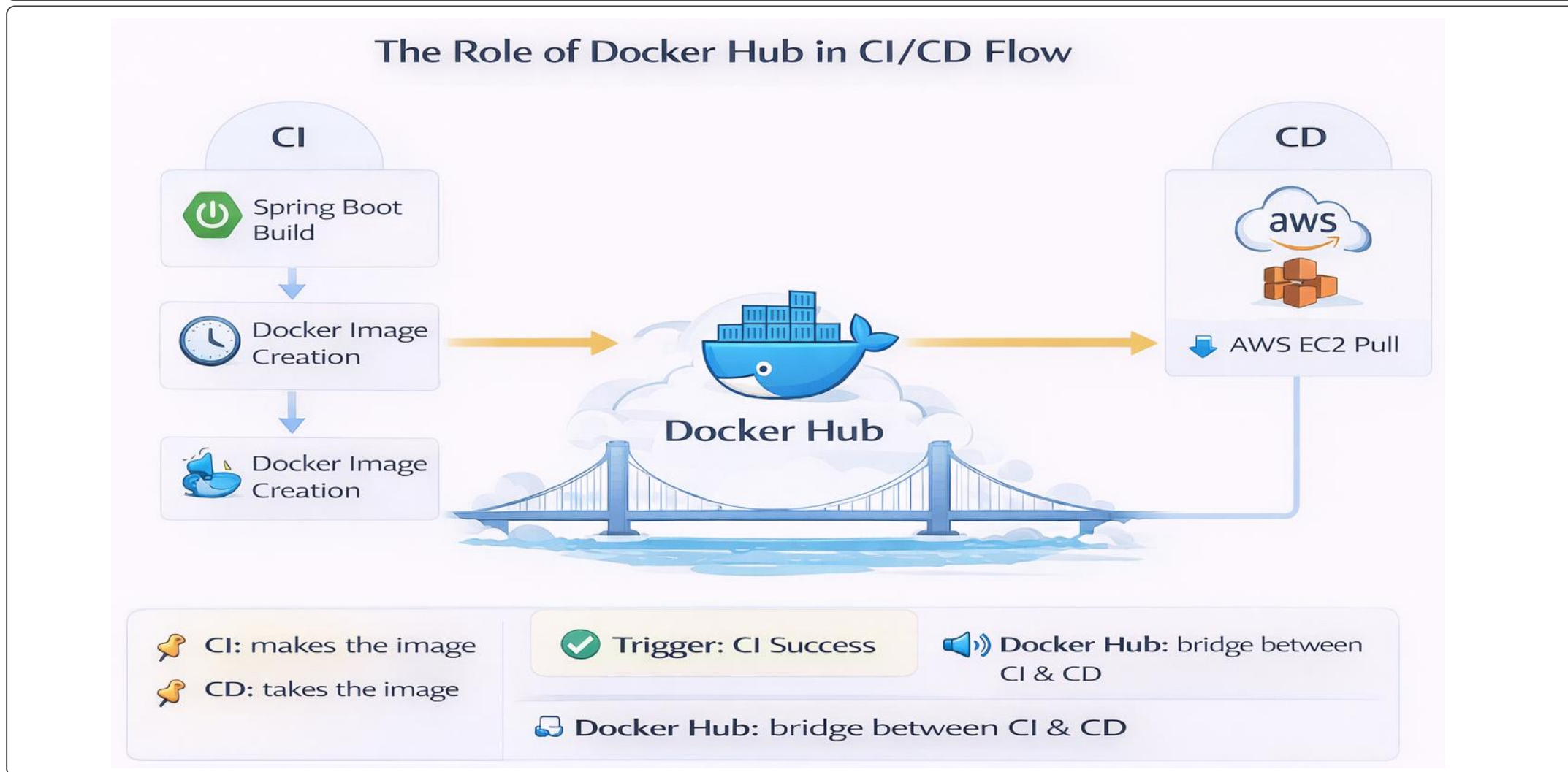
Docker Hub



# ≡ Docker Hub

## ≡ Docker Hub 저장소

CI가 만든 결과물을 저장하고, CD가 그 결과물을 가져 가는 중간 저장소 역할을 합니다.





# ≡ Docker Hub

## ≡ Docker Hub 저장소

Docker Hub에서 Build 결과물이 저장될 저장소를 준비합니다.

Repositories / [cicd-springboot](#) / General

Using 0 of 1 private repositories.

**ugcadman/cicd-springboot** ⓘ

Created 1 minute ago · ☆ 0 · ↓ 0

Spring Boot CI/CD Test ⓘ

Add a category ⓘ ⓘ

**Docker commands** [Public view](#)

To push a new tag to this repository:

```
docker push ugcadman/cicd-springboot:tagname
```

메모장에 기록하기  
형식 : 계정아이디/Repository이름

General Tags Image Management Collaborators Webhooks Settings

Tags ⓘ INCOMPLETE

Pushed images appear here.

Repository 생성

DOCKER SCOUT INACTIVE [Activate](#)

buildcloud

Build with

Repository overview ⓘ

An overview describes what you can expect from your repository in the public view of your repository.

[Add overview](#)

"<https://hub.docker.com/repositories/>개발자\_계정"에서 저장소(Repository)를 생성합니다.

※ 해당 정보는 GitHub 서버에서 Docker Hub로 이미지 업로드할때 사용합니다.

Docker Hub 저장소는 CI(빌드) 결과물인 "Docker 이미지"를 CD(배포) 단계로 전달해 주는 "중간 창고" 역할을 합니다. GitHub Repo의 Docker 이미지 버전이라고 볼 수 있습니다.

## ≡ Docker Hub Access Token

Docker Hub Access Token은 "비밀 번호 대신 사용하는 CI/CD 전용 인증 키" 역할을 합니다.

사용 이유

### Why Use Docker Hub Access Token?

#### Docker Hub Account Password

-  Full Account Access
-  Account Takeover Risk
-  Breaks All Automation on Change

VS



VS

#### Access Token

-  For CI/CD Pipelines
-  Restricted Permissions
-  Easy to Revoke Anytime

# ≡ Docker Hub

## ≡ Docker Hub Access Token

Access Token은 소스 코드에 존재하지 않고, GitHub Secrets라는 곳에 저장되어 있습니다.

### Access Token Location in CI/CD



Not in Code



Stored in GitHub Secrets



Injected Only on Run

## Docker Hub Access Token

Access Token의 장점은 '자동화에 최적화', '권한 분리', '보안 사고 대응 쉬움'입니다.

### Benefits of Access Tokens (CI/CD Perspective)

#### Optimized for Automation



Easily Expire & Regenerate



Not Affected by PW Changes

#### Granular Permissions



Read-Only (Pull Only)



Write Access  
(Push Capable)

#### Easy to Respond to Incidents



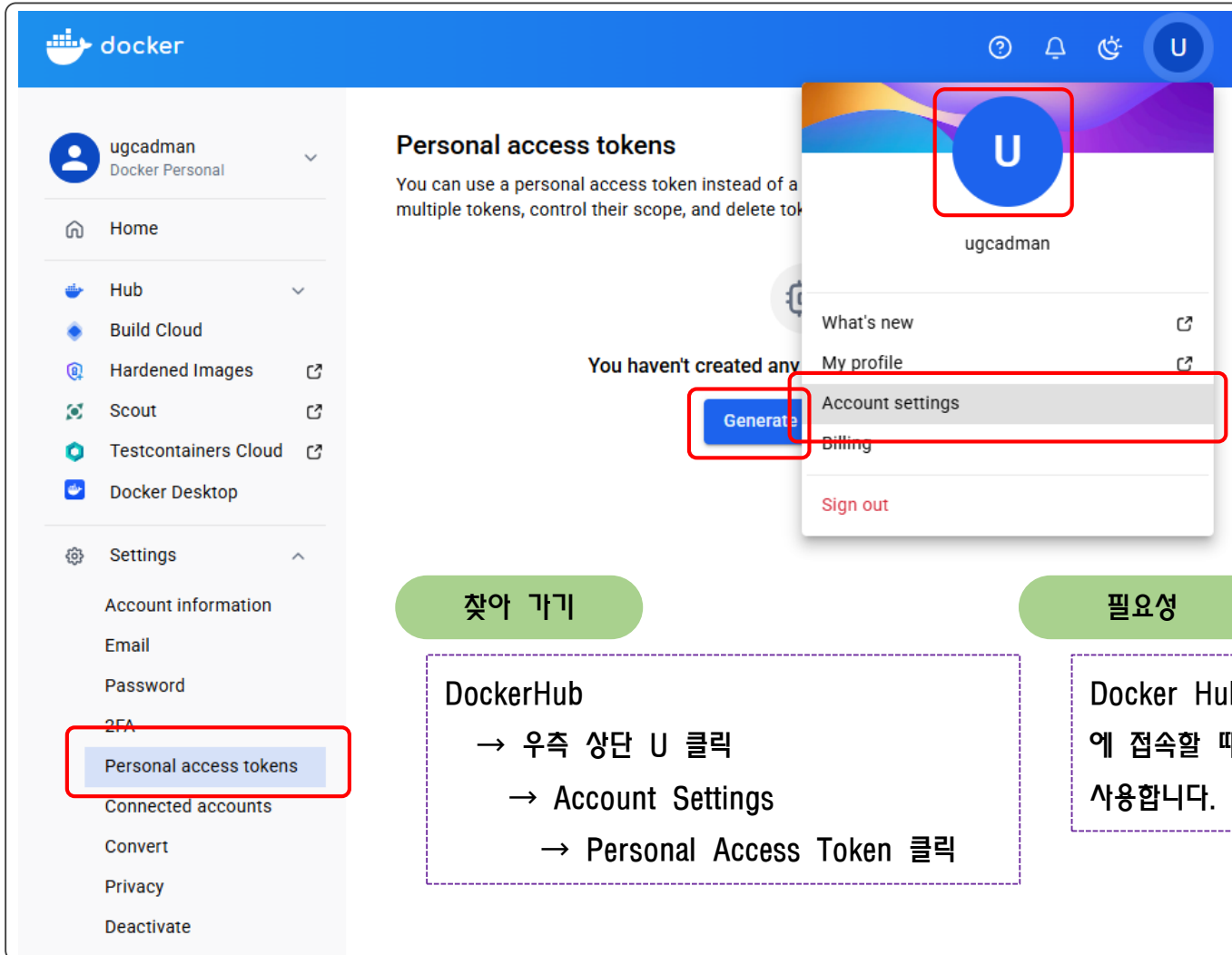
Revoke a Single Token



No Service-Wide Impact

## Docker Hub Access Token

DockerHub에서 Personal Access Token을 생성합니다.



찾아 가기

DockerHub

→ 우측 상단 U 클릭

→ Account Settings

→ Personal Access Token 클릭

필요성

Docker Hub Access Token은 CI/CD에서 나의 Docker Hub 계정에 접속할 때 "비밀번호 노출 위험"에 대한 부담을 덜어 내기 위하여 사용합니다.

## Docker Hub Access Token

DockerHub에서 Personal Access Token을 생성합니다.

### Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#) 

Access token description

github-actions

Access token description : github-actions

Expiration date

None

Expiration date : None

Optional

Access permissions

Read & Write

Access permissions: Read / Write

Read & Write tokens allow you to p

Generate 버튼 클릭

Cancel

Generate



## Docker Hub Access Token

DockerHub에서 Personal Access Token을 생성합니다.

### Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#) 

Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

#### Access token description

github-actions

#### Expires on

Never

#### Access permissions

Read & Write

To use the access token from your Docker CLI client:

1. Run

```
$ docker login -u ugcadman
```

Copy

2. At the password prompt, enter the personal access token.

dckr\_pa

Copy

Access token 정보를  
메모장에 복사해 둡니다.

# CI/CD

GitHub Secret



# ≡ GitHub Secret

## ≡ GitHub Secret 개요

Source code에 절대 들어가면 안 되는 **민감 정보를 안전하게 저장**하고, 실행 시점에만 사용하는 **보안 저장소**입니다.



### Common Values Stored in GitHub Secrets



Access Tokens (Docker Hub, Cloud, API)



Passwords



API Keys



Private Keys

Managed at Repository / Environment /



Repository



Environment



Organization

## 왜 필요한가

Code에 직접 넣는 경우와 GitHub Secrets에 추가하는 경우의 차이점을 살펴 보겠습니다.

### Why Use GitHub Secrets?

#### Case A. Directly Entering Code



Permanently Recorded in Git History



Exposed on Fork / Clone



Accidentally Pushed → Immediate Leak



```
docker login \  
-u my_username  
-p my_docker_hub_access_token
```

VS

#### Case B. Using GitHub Secrets



Only Variable Names in Code



Actual Values Stored Securely Inside GitHub



Injected Only During Execution



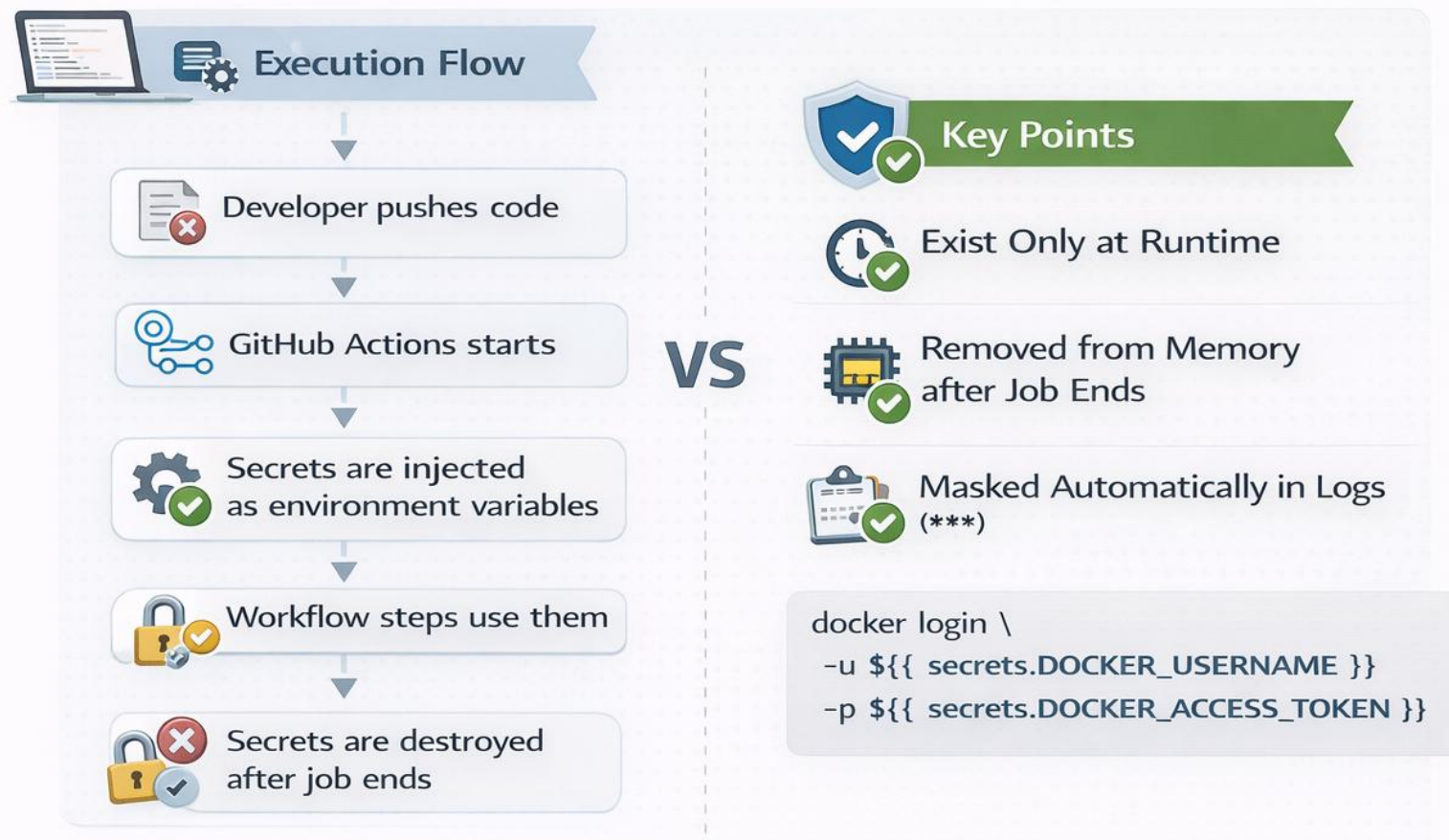
```
docker login \  
-u ${ secrets.DOCKER_USERNAME }  
-p ${ secrets.DOCKER_ACCESS_TOKEN }
```

# GitHub Secret

## 동작 방식

소스 코드가 Push될때, GitHub Actions가 동작하면서 Secrets내의 값을 참조한 이후 작업 종료시 자동으로 소진됩니다.

### How GitHub Secrets Work





## 보안상의 특징

암호화(Encryption)로 저장되고, Masking이 가능하며, 사용 범위 제어(Scope Control)가 가능합니다.

### Key Security Features of GitHub Secrets



# ≡ GitHub Secret

## ≡ GitHub Secrets vs Environment Variables

GitHub Secrets와 환경 변수(Environment Variables)를 비교해 봅니다.

### GitHub Secrets vs Environment Variables

| Item                                                                                                                                                                                                                |  GitHub Secrets      |  Environment Variables (Env)  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  Storage Location                                                                                                                  |  GitHub secure store | Code / Server                                                                                                                                                                                       |
|  Encryption                                                                                                                        | O                                                                                                       | X                                                                                                                                                                                                   |
|  Log Masking                                                                                                                       | O                                                                                                       | X                                                                                                                                                                                                   |
|  Access Control                                                                                                                    | O                                                                                                       |  Limited                                                                                                         |
|  CI/CD Specific                                                                                                                  | O                                                                                                       | X                                                                                                                                                                                                   |
|  CI/CD  ALCmate / Staging Environment Updates |                                                                                                         |                                                                                                                                                                                                     |
|  Automate Production / Staging Environment Updates                                                                               |                                                                                                         |                                                                                                                                                                                                     |



## ≡ Best Practices

시크릿 관리는 '편의성'이 아니라 '사고 범위'를 줄이기 위한 설계입니다

### CI/CD Secrets Best Practices



One Secret = One Purpose



Apply Least Privilege



Separate by Env/Purpose



No Logging Output



Never Commit to Git!

